Code Authorship Detection

SemEval-2026 Task 13 - Subtask A

Model and experiment

This project fine-tunes microsoft/codebert-base to classify source code as human-written (label 0) or machine-generated (label 1). CodeBERT pretraining alone does not provide a trained authorship classifier. The application loads only a trained local checkpoint.

The recorded run used 2,000 training snippets and 500 validation snippets, with random seed 42, 500 optimization steps and a maximum token length of 256. Best checkpoint selection uses validation macro F1. Smoke run: False.

Selected validation macro F1: 0.9579. Selected checkpoint: checkpoint-500.

Data and methodology

Source: DaniilOr/SemEval-2026-Task13, configuration A. Training and validation examples are sampled with class stratification from their respective official splits. Identical trimmed snippets are removed across splits. All model fitting uses training data; validation selects the checkpoint, and test data is used only for final evaluation. Training, inference and evaluation share the same language-prefix preprocessing.

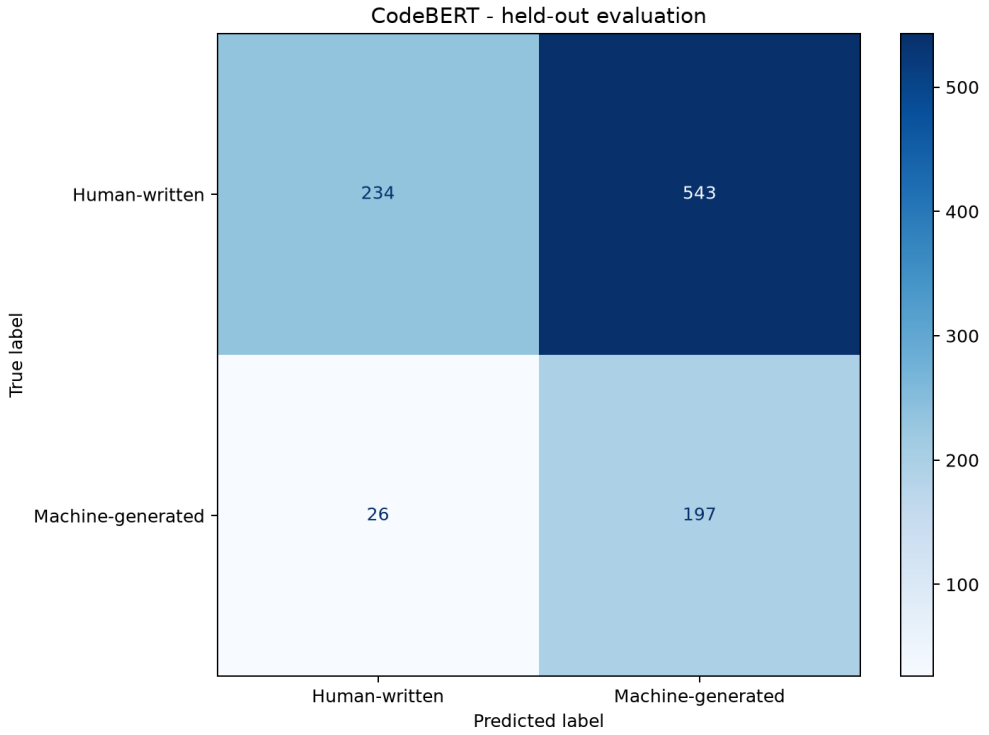
The test set is the organizer-provided public sample of the test data, not the full competition test set. This initial experiment uses a small training subset and must not be presented as a full-data or leaderboard result.

Held-out results

Test samples: 1,000. Accuracy: 43.10%. Macro precision: 0.5831; macro recall: 0.5923; macro F1: 0.4302.

For context, always predicting the majority class gives 77.70% accuracy on this sample. Macro F1 and per-class recall should be considered alongside accuracy because the class distribution is uneven.

Class	Precision	Recall	F1	Support
Human-written	0.9000	0.3012	0.4513	777
Machine-generated	0.2662	0.8834	0.4091	223



Interpretation

The measured test accuracy is below the majority-class accuracy baseline. Despite strong validation scores, this initial model generalizes poorly and should not be presented as a reliable authorship detector. The training subset contains 1,825 Python examples out of 2,000, while the public test sample covers eight languages. Distribution shift may contribute to the gap, but its causal contribution has not been isolated. Future improvements should be selected using validation data and assessed on a fresh held-out set.

Application and limitations

The Gradio app accepts Python, C++ and Java, displays model probabilities, and uses Lizard for per-function cyclomatic complexity. Complexity measures independent control-flow paths; it does not establish authorship or Big-O runtime. Probabilities are not calibrated certainty and should not be used as proof of misconduct. Long code is truncated. Exact deduplication cannot rule out near-duplicate solutions or shared programming problems.

Reproducibility

Run prepare_data.py, train_model.py, evaluate_model.py, then generate_report.py. See README.md for exact commands and subset sizes. Data manifests and training metadata record random seeds, row counts and SHA-256 hashes. Results include a per-example CSV, machine-readable metrics and a confusion matrix.

References

Official task: https://github.com/mbzuai-nlp/SemEval-2026-Task13
Official dataset: https://huggingface.co/datasets/DaniilOr/SemEval-2026-Task13
Base model: https://huggingface.co/microsoft/codebert-base